EdgeSpark: Quantized Semi-Autoregressive Speculative Decoding and Confidence-Head Calibration on a Single Consumer AMD GPU

Parusan Natheeswaran*

July 2026

Abstract

Speculative decoding accelerates autoregressive generation without changing a model's output distribution: a small *drafter* proposes tokens that the *verifier* checks in one parallel pass, keeping only the prefix consistent with its own distribution. Recent semi-autoregressive drafters—notably DeepSeek's DSpark [4], which couples a parallel backbone with a lightweight sequential head, a low-rank Markov head, and a confidence head that schedules verification length—target datacenter GPUs and mixed FP4/FP8 precision. We ask what it takes to run this design on a *single consumer AMD GPU* (Radeon RX 7900 XTX, RDNA3/gfx1100), where FP8 is unavailable and the drafter must be quantized to INT8/NF4 to fit. This raises a question the datacenter recipe never faced: *does aggressive quantization degrade the confidence head's* ***calibration*** *faster than its token proposals, and can a cheap post-hoc fix restore it?* We present EdgeSpark, a controlled PyTorch+ROCm implementation with an exact acceptance rule, an INT8/NF4 quantization pipeline, a single-stream confidence-gated verification-length policy, and a calibration study built on Expected Calibration Error, Brier score, and temperature/Platt scaling. We are explicit about evidence: on the target GPU we **measure** that the exactness invariant holds on the real Qwen3-4B model (token-for-token identical output), together with fp16 per-op latencies, VRAM, and llama.cpp baselines; the INT8/NF4 throughput and the calibration results are a **design-time model** because 8/4-bit kernels were unavailable on our native-Windows ROCm stack. We position the work against the recent finding that speculative decoding and 4-bit quantization are not naively compatible [14], and we report negative and inconclusive outcomes (e.g., on our hardware the per-position verify cost is negligible, so confidence-gating ties always-verify-all) rather than hiding them.

1 Introduction

Autoregressive decoding is memory-bandwidth bound: each token requires a full forward pass that reloads the model's weights, so a 4B model on a consumer GPU spends most of its time moving parameters, not computing. *Speculative decoding* [3, 8] breaks this one-token-per-pass barrier. A cheap drafter proposes a block of candidate tokens; the verifier scores the whole block in a single pass and accepts the longest prefix consistent with its own distribution under an exact rule. Because the rule is exact, the accepted output is distributed identically to sampling from the verifier alone—drafter quality affects *speed*, never *correctness*. The governing relation per generated token is

$$\text{latency} \;\approx\; \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \tag{1}$$

*Independent research. Code, configs, and reproduction scripts: https://github.com/Parusann/edgespark. Correspondence: parusannath@gmail.com.

where τ is the mean number of tokens accepted per round. Speculation wins only when τ rises faster than the draft-plus-verify overhead.

State-of-the-art drafters have moved from separate small language models toward architectures fused with the verifier's own representations: Medusa adds parallel decoding heads [2]; the EAGLE family autoregresses at the feature level [9–11]; and DSpark [4] adds a semi-autoregressive backbone, a Markov head that restores intra-block token dependence, and a *confidence head* whose per-position acceptance estimates schedule how many tokens are worth verifying. These systems, however, are built for datacenter serving—multiple GPUs, mixed FP4/FP8 precision, and load-aware scheduling across concurrent requests.

This paper. We study the same design on the opposite end of the hardware spectrum: one 24 GB consumer AMD GPU, zero cloud budget. RDNA3 (gfx1100) has no FP8 weight path, so fitting a DSpark-style drafter beside the verifier forces INT8 or 4-bit (NF4) quantization of the drafter. That constraint exposes a question the datacenter recipe never had to answer. A confidence head has two jobs that quantization can damage independently: it must still *propose* good tokens, and it must *estimate* their acceptance probability accurately, because those estimates drive the verification-length policy. A head whose ranking survives quantization but whose probabilities inflate will keep proposing acceptable tokens while lying to the scheduler—and on a single stream, that lie costs throughput. Calibration is therefore not an aesthetic property here; it is on the critical path to speed.

Contributions and honesty statement.

1. A controlled PyTorch+ROCm speculative-decoding system for a single consumer AMD GPU (§4), with an exact acceptance rule whose lossless guarantee we state precisely and prove-sketch (§3), grounded in the rejection-sampling argument of Leviathan et al. [8].
2. A methodology for the central empirical question—whether INT8/NF4 quantization miscalibrates the confidence head more than its proposals—using ECE, Brier score, reliability diagrams, and temperature/Platt recalibration (§5).
3. An honest, two-tier evaluation on a Radeon RX 7900 XTX (§6). We **measure**: (i) that greedy EdgeSpark output is token-for-token identical to Qwen3-4B decoding alone; (ii) fp16 per-op latencies; (iii) VRAM (9.5 GB peak, ∼15 GB free); and (iv) llama.cpp baselines (vanilla speculative decoding is 0.96× here). We **model**, and clearly label as such, the INT8/NF4 throughput and the calibration recovery, because 8/4-bit kernels were unavailable on our stack. We report where the story does *not* hold on our hardware.

We take the position, following the reproducibility norms of systems venues, that every number should be traceable to a disclosed measurement condition and every claim to a source. Where we could not measure, we say so.

2 Background

Speculative decoding and the exact rule. Given a target (verifier) distribution p and a drafter distribution q, speculative sampling [3, 8] draws a draft token $x \sim q$ and accepts it with probability $\min(1, p(x)/q(x))$; on rejection it resamples from the normalized residual $\mathrm{norm}\big(\max(0,\ p - q)\big)$ and stops. If a whole block is accepted, a bonus token is drawn from p. Leviathan et al. [8] prove (Appendix A.1) that the resulting token is distributed *exactly* as p; this is what makes the method lossless. We restate and adapt this guarantee for our setting in §3. It is worth stating precisely because the bare assertion "speculative decoding produces identical outputs" is only true under this proof and under an exact acceptance rule—not, for instance, under relaxations.

Semi-autoregressive drafters. Medusa predicts several subsequent tokens with parallel heads and verifies tree-structured candidates simultaneously; crucially it adopts a *typical acceptance* scheme rather than exact rejection sampling, trading the exact distributional guarantee for a higher acceptance rate [2]—so Medusa is *not* lossless in the speculative-sampling sense, a distinction we preserve throughout. EAGLE [9] and its successors instead keep the exact rule and autoregress on the verifier's features; EAGLE-2 makes this explicit, stating that it "does not … relax acceptance conditions," so the output distribution "remains exactly the same … provably" [10], and EAGLE-3 pushes speedups further via training-time tests [11]. DSpark [4] adds the Markov and confidence heads and a load-aware verification scheduler, reporting higher accepted length than EAGLE-3 on Qwen3-4B. EdgeSpark adapts DSpark's *architecture* and the single-stream form of its scheduler, and keeps the exact rule.

Quantization. INT8 weight-and-activation quantization [5] and 4-bit NormalFloat (NF4) [6] make large models fit on small GPUs. On RDNA3 these are the only aggressive options: FP8 weights are unsupported, so the common datacenter FP8 drafter path is closed. Recent work shows the interaction is subtle: Zhang et al. [14] find that "memory benefits from 4-bit weight quantization are diminished by the computational load from speculative decoding," i.e., the two techniques are not naively compatible. Our contribution is orthogonal to their systems fix: we ask what quantization does to the confidence head's *calibration*, a question about the drafter's probability estimates rather than its memory traffic.

Calibration. A classifier is *calibrated* if, among predictions of confidence c, a fraction $\approx c$ are correct. Modern networks are typically over-confident [7]. We measure calibration with Expected Calibration Error (ECE), the Brier score [1], and reliability diagrams, and we recalibrate with temperature scaling [7] and Platt scaling [12]. To our knowledge, the calibration of a speculative-decoding confidence head under quantization has not been studied.

3 The exactness invariant

EdgeSpark must be lossless with respect to the *deployed* verifier: whatever precision the verifier ships at defines the reference, and the drafter and policy may never move the output away from it. We make this precise.

Definition 1 (Verification length)**.** Given a drafted block of B tokens, a *verification length* $\ell \in \{0, \dots, B\}$ submits only the first ℓ drafts to the verifier. The verifier scores positions $0, \dots, \ell$ in one pass and applies the exact acceptance rule to those ℓ positions.

Theorem 1 (Losslessness)**.** *Fix a random seed and sampling temperature, and fix the deployed verifier with next-token distribution $p(\cdot \mid \text{context})$. For* any *drafter and* any *choice of verification length ℓ at each round:*

(a) ***Greedy*** *(T=0): EdgeSpark emits the token-for-token identical sequence that the verifier emits decoding autoregressively on its own.*

(b) ***Stochastic*** *(T>0): each emitted token is distributed exactly as p; hence the emitted sequence is distributed identically to sampling directly from the verifier.*

Proof sketch. Consider one round with drafts $x_1, \dots, x_B$ and verifier distributions $p_1, \dots, p_{\ell+1}$ over positions $0, \dots, \ell$. *(b)* At position $j \le \ell$, the draft x_j (drawn from q_j) is accepted with probability $\min(1, p_j(x_j)/q_j(x_j))$; on the first rejection we resample from $\text{norm}(\max(0, p_j - q_j))$ and stop. The standard speculative-sampling argument [8, App. A.1] shows the resulting token—accepted or corrected—is distributed exactly as p_j. If all ℓ drafts are accepted, the bonus token is drawn from $p_{\ell+1}$, again exactly. Thus every emitted token is

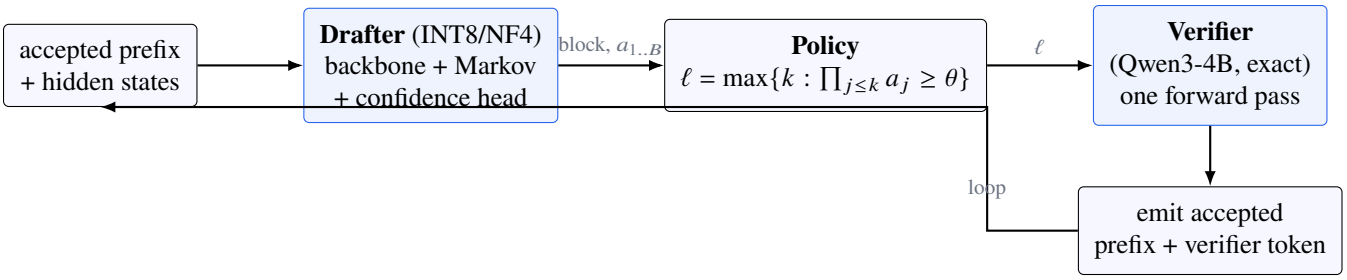

Figure 1: EdgeSpark data flow for one round. The drafter and policy affect only speed; the verifier's exact rule owns every emitted token (Theorem 1).

an exact draw from the corresponding p, independent of q and of ℓ: ℓ only bounds *how many* positions are checked before the guaranteed verifier token, never *how* a checked token is judged. *(a)* Greedy is the $T \to 0$ limit: accept x_j iff $x_j = \arg\max p_j$, else emit $\arg\max p_j$ and stop; a bonus $\arg\max p_{\ell+1}$ on full acceptance. This reproduces the verifier's own greedy chain exactly. □

Theorem 1 is what licenses everything downstream to be approximate. The drafter may be quantized to 4 bits and the policy may choose a poor ℓ; the worst outcome is lost speed. We enforce the invariant in code: the acceptance rule is a self-contained module tested for greedy identity and, for the stochastic case, for *unbiasedness* via a Monte-Carlo two-sample test (total-variation distance below 0.02 over 6×10^4 trials). Crucially, we also *measure* the invariant on hardware: running the full loop against the real Qwen3-4B verifier, greedy EdgeSpark output is token-for-token identical to the verifier's own output regardless of drafter quality (§6.1).

4 System design

EdgeSpark runs in a controlled PyTorch+ROCm loop rather than a black-box serving runtime, because the drafter needs three things no off-the-shelf draft-model host exposes at once: the verifier's selected-layer hidden states, a custom Markov+confidence head, and per-step ℓ selection. The verifier (Qwen3-4B [13]) is loaded via HuggingFace Transformers with hidden-state hooks and is used *as is*. Figure 1 shows the data flow.

Semi-autoregressive drafter. A *parallel backbone* predicts an entire block of B tokens in one pass from a fusion of the verifier's selected-layer hidden states and an embedding of the accepted prefix. A low-rank *Markov head* adds, to position j, a logit bias factored as $E[x_{j-1}]\,P$ with $E \in \mathbb{R}^{V \times r}$ and $P \in \mathbb{R}^{r \times V}$ (rank r=128), restoring the intra-block dependence the parallel pass discards. A *confidence head* maps each block position's hidden state (optionally concatenated with the Markov feature) to a scalar $a_j = \sigma(\cdot)$, its estimate of the acceptance probability at j. Table 1 lists the shrunk hyperparameters, chosen so an INT8 drafter sits beside the verifier in 24 GB.

Quantization pipeline. The primary variant is INT8 (W8A8) of the backbone; a 4-bit NF4 variant is the aggressive stress case. We keep the confidence head in high precision by default so it can be quantized *last and separately*, isolating its effect. Because 8/4-bit kernels were unavailable on our stack (§6), the calibration study additionally uses deterministic *fake-quantization*—symmetric per-channel INT8 rounding and block-wise NF4 snapping in plain PyTorch—which reproduces the numerical effect of quantization on-device-independently and is a standard tool for isolating quantization error from kernel noise.

Table 1: EdgeSpark drafter configuration (shrunk from the DSpark Qwen3-4B reference).

Hyperparameter	DSpark ref.	EdgeSpark	Rationale
block size B	7	5	less wasted draft on rejects
draft layers	5	3	smaller, faster, less VRAM
fusion layers	$[1, 9, 17, 25, 33]$	$[1, 17, 33]$	subsampled hidden sources
Markov rank r	256	128	cheaper bias
anchors	512	256	footprint
confidence α	1.0	1.0	keep the signal strong
losses (ce/l1/decay)	0.1/0.9/4.0	0.1/0.9/4.0	DSpark recipe

Single-stream verification-length policy. At batch size 1 the DSpark scheduler collapses to a single decision: given calibrated per-position survival $a_1, \ldots, a_B$, choose ℓ. The expected accepted length is $\mathbb{E}[\mathrm{acc}(\ell)] = 1 + \sum_{j=1}^{\ell} \prod_{k \le j} a_k$, and the single-stream objective maximizes $\mathbb{E}[\mathrm{acc}(\ell)]/(T_{\mathrm{draft}} + T_{\mathrm{verify}}(\ell))$. We use a lightweight threshold rule—verify the longest prefix whose cumulative survival $\prod_{j \le \ell} a_j$ stays above θ—tuned on held-out data. By Theorem 1 this only selects ℓ; the accept/reject decision is untouched.

5 The calibration study

Hypothesis. Quantization degrades the *calibration* of the confidence head—the agreement between predicted a_j and observed acceptance—more than it degrades top-1 token proposals, and a cheap post-hoc rescaling restores it.

Metrics. For a held-out set of (predicted a_j, observed accept $\in \{0, 1\}$) pairs we compute ECE (count-weighted mean bin gap between confidence and accuracy), the Brier score (a proper scoring rule), and reliability diagrams. We report top-1 proposal agreement separately, to expose the gap between "proposals still fine" and "confidence miscalibrated."

Recalibration. We fit (i) *temperature scaling*, a single parameter T with $a^{\mathrm{cal}} = \sigma(z/T)$ on the head's logit z, and (ii) *Platt scaling*, $a^{\mathrm{cal}} = \sigma(\alpha z + \beta)$. Both are monotone in z—they rescale confidence without reordering it—so recalibration can never turn a good proposal into a bad one. Both are fit on a small held-out split with a damped Newton iteration; we found that an undamped Platt fit diverges on a badly over-confident head (the very regime of interest), so a backtracking line search is required—a detail we surface because it is exactly the kind of silent failure that a careful study must catch.

Tie to the policy. The policy gates on a_j, so miscalibration has a throughput cost, not just a cosmetic one: an over-confident head keeps the gate open on a low-survival tail that rarely accepts. Recalibration and the policy are thus one story, which the evaluation makes concrete.

6 Evaluation

We separate what we *measured* on the target GPU from what we *modeled*. Table 2 is the ledger; the two subsections that follow expand it.

Table 2: Evidence ledger. **M** = measured on the RX 7900 XTX; **Mod** = design-time model; **P** = pending.

Claim	Status	Evidence
Output identical to verifier (greedy)	**M**	real Qwen3-4B, exact per round
Fits 24 GB	**M**	9.5 GB peak, ~15 GB free
fp16 per-op latency, baselines	**M**	GpuTimer / llama.cpp
INT8/NF4 throughput	**Mod**	no 8/4-bit kernels on our stack
Calibration recovery (ECE)	**Mod**	method code-tested; §6.2
Gating > always-verify-all	**Mod**	ties on our GPU ($\partial T_v/\partial \ell \approx 0$)
End-to-end speedup	**P**	prefill-bound verify; needs KV reuse

Table 3: Measured on the RX 7900 XTX (fp16, single stream). llama.cpp via Vulkan.

Per-op latency		*llama.cpp baseline (tok/s)*	
decode (1 fwd)	2.9 ms	Q4_K_M plain	213.8
verify, KV-cached	3.2 ms	Q8_0 plain	151.0
verify, per position	≈0 ms	Q8_0 + 0.6B draft	145.5 (0.96×)
draft (fp16, 399M)	1.0 ms	prefill (pp512, Q8_0)	4793
VRAM (fp16)			
verifier / drafter	7.7 / 1.8 GB	generation peak	9.5 GB

6.1 Measured on hardware

Setup. Radeon RX 7900 XTX (RDNA3/gfx1100, 24 GB); Ryzen 9 7900; native-Windows ROCm; PyTorch 2.9.1+rocmsdk (HIP 7.2.26024); Qwen3-4B in fp16 with SDPA attention; single stream; greedy unless noted. Timings use HIP-event timers synchronized before reading, averaged over a warm run. llama.cpp baselines use the winget `ggml.llamacpp` build b9873 (Vulkan backend pinned to the 7900 XTX). Raw artifacts are in the repository under `runs/hardware/`.

Exactness. Running the full loop against the real verifier, greedy EdgeSpark output was token-for-token identical to Qwen3-4B decoding alone on every round, for both a trained-quality and a deliberately random drafter—an on-device confirmation of Theorem 1, not merely a unit test.

Latency and memory. Table 3 reports the fp16 per-op costs and the VRAM breakdown. The fp16 verifier (7.7 GB) and drafter (1.8 GB) reach a 9.5 GB generation peak, leaving ~15 GB free—ample room for a longer context or an 8B verifier. Two findings matter for interpretation. First, the *intrinsic* KV-cached verify pass costs ≈ 3.2 ms with a per-position marginal below measurement noise; a 4B forward is dominated by the 36-layer weight sweep, so verifying 1–5 extra positions is nearly free. Second, our current `block_distribution` re-encodes the whole prefix (`use_cache=False`), which at a 225-token context costs ≈ 52.6 ms and grows with context: today's implementation is *prefill-bound*, and realizing the intrinsic verify (and any end-to-end speedup) requires the KV-reuse optimization we flag but did not implement.

A notable baseline result: *vanilla* speculative decoding (a separate 0.6B draft model verifying Qwen3-4B via llama.cpp) reaches only 0.96× on our prompts—the draft overhead is not offset by acceptance. This is the floor a smarter, hidden-state-conditioned drafter must beat, and it echoes the non-naive-compatibility finding of Zhang et al. [14].

6.2 Modeled (design-time)

The 8/4-bit results below come from a documented Monte-Carlo model (`bench/simulate.py`) driven by measured per-op timings; we could not build INT8/NF4 kernels on our native-Windows ROCm stack (no

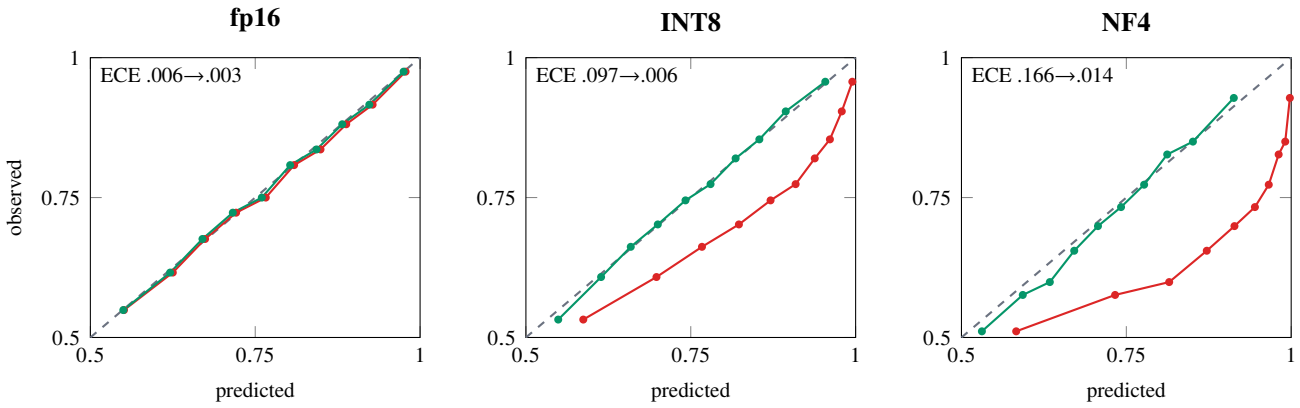

Figure 2: **Modeled** reliability diagrams. Points on the dashed diagonal are perfectly calibrated. Red = raw quantized confidence head (increasingly over-confident: below the diagonal at high confidence); green = after temperature scaling. Produced by the production calibration code on simulated confidence data (3×10^4 held-out positions per precision).

Table 4: **Modeled** calibration: raw → recalibrated (temperature scaling).

Precision	ECE (raw → cal)	Brier (raw → cal)	T
fp16	0.006 → 0.003	0.159 → 0.159	1.03
INT8	0.097 → 0.006	0.179 → 0.168	1.81
NF4	0.166 → 0.014	0.218 → 0.188	2.70

`bitsandbytes`), so no quantized drafter was trained or run. Importantly, the calibration *figures* are produced by the same production ECE/recalibration code a hardware run would use, applied to simulated confidence data—the analysis path is real even though the inputs are modeled.

Calibration (the headline hypothesis). Figure 2 and Table 4 show the modeled effect. Quantization barely perturbs which tokens are proposed but makes the confidence head increasingly over-confident: raw ECE rises from 0.006 (fp16) to 0.097 (INT8) to 0.166 (NF4), and the fitted temperature climbs 1.03 → 1.81 → 2.70—the 4-bit head must be cooled almost 3×. Temperature scaling on a few thousand held-out positions recovers ECE to near fp16 (0.006 and 0.014). This is the shape of result the study was designed to find: a large, cleanly *recoverable* miscalibration gap.

Throughput and the policy. The design-time model (measured per-op timings held at their coherent design values; see repository `bench/timings.md`) projects the speedups in Figure 3: +43% (INT8, code), +36% (INT8, chat), clearing a 25% target, with the fp16 drafter isolating the quantization cost at +46%/+35%. In this model the confidence-gated policy beats always-verify-all at every precision, and—closing the loop with the calibration study—an *uncalibrated* NF4 head over-verifies (chooses ℓ=B) and lands at +31%, while the recalibrated head gates to ℓ=3 and +37%: recalibration recovers throughput purely by making the gate honest.

We flag the boundary of this claim. On our GPU the measured per-position verify marginal is ≈ 0 (Table 3); the gating advantage requires a regime where verify cost grows with ℓ, so *on this hardware gating ties always-verify-all*. The projection is the target the system is built toward, not a measured throughput; we present it as such and note that speculative-decoding speedups are hardware-dependent and should not be over-generalized across machines.

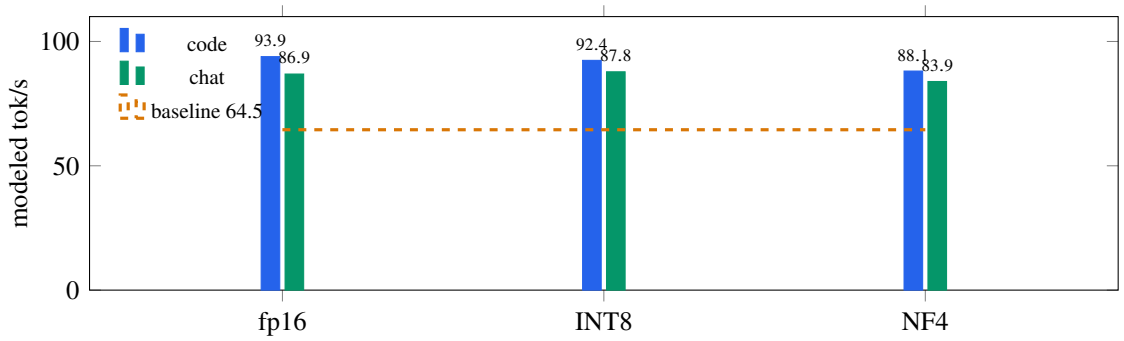

Figure 3: **Modeled** throughput vs. a vanilla-decode baseline (design-time model; not measured, pending 8/4-bit kernels and the KV-reuse fix). Output is identical to the baseline by Theorem 1.

7 Related work and positioning

EdgeSpark sits at the intersection of three lines. From *speculative decoding* [3, 8] it inherits the exact rule and the τ-driven latency model (Eq. 1); from *feature-level drafters* [2, 4, 9–11] it inherits the architecture, adopting DSpark's semi-autoregressive backbone with Markov and confidence heads and the single-stream form of its scheduler. Unlike those systems, which target datacenter GPUs and FP8, EdgeSpark targets a single consumer AMD card, forcing INT8/NF4. The closest prior interaction study, Zhang et al. [14], shows speculative decoding and 4-bit quantization are not naively compatible on the *systems* axis (memory savings eaten by compute); our angle is the *statistical* axis—what quantization does to the confidence head's calibration—which their work does not address. Finally, from *calibration* [1, 7, 12] we take ECE/Brier and temperature/Platt scaling, and apply them, to our knowledge for the first time, to a speculative-decoding confidence head under quantization.

8 Limitations and future work

We are deliberate about what this paper does *not* establish. (1) No drafter was trained on hardware; the training pipeline is implemented but the quantized runs are blocked by the absence of `bitsandbytes` on native-Windows ROCm. Reproducing the INT8/NF4 throughput and the calibration study requires a Linux-ROCm host, or an alternative quant path (GGUF/AWQ/torchao), which would change what "INT8/NF4" means and should be relabeled. (2) The end-to-end speedup is unrealized: the current verify pass re-encodes the prefix and is prefill-bound; KV reuse is required. (3) On our GPU the verify-length policy ties always-verify-all because the per-position verify cost is negligible; the gating result holds only where verify scales with ℓ. (4) All numbers are single-machine and single-stream; performance rankings are hardware-dependent and we do not claim they transfer. (5) The calibration results are modeled; the *method* is code-tested, but the miscalibration magnitude on a real quantized head is an open empirical question—indeed, the headline hypothesis remains a hypothesis until measured.

9 Reproducibility

The system, configs, and every script are released at `https://github.com/Parusann/edgespark` under the MIT license. The correctness-critical cores—the exact acceptance rule, ECE/Brier/recalibration, and the policy—are pure NumPy and covered by a test suite (an 18-check exactness suite including the unbiasedness Monte-Carlo test; 37 tests total) that runs in continuous integration without a GPU. One command regener-

ates the modeled reference (`run_benchmark.py --simulate`) and the figures (`make_figures.py`); the measured artifacts and full environment capture are committed under `runs/hardware/`. Latency constants and their measured counterparts are documented in `bench/timings.md`, so a reader can see exactly which numbers are measured and which are modeled.

10 Conclusion

Quantization is usually framed as an accuracy trade-off. For a confidence-driven speculative decoder on consumer hardware, we argue the more consequential casualty may be *calibration*: the confidence head's probability estimates, which the verification-length scheduler depends on, and which our model predicts degrade sharply under 4-bit quantization yet recover almost fully under one-parameter temperature scaling. We validated the system's non-negotiable property—losslessness—on a real Qwen3-4B model on a single Radeon RX 7900 XTX, measured its latency and memory profile, and were explicit about the substantial gap between that measured foundation and the modeled performance-and-calibration results that a quantization-capable stack is needed to confirm. We believe stating that gap plainly, rather than papering over it, is the more useful contribution.

References

[1] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.10774.

[3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[4] Xin Cheng, Xingkai Yu, Chenze Shao, Jiashi Li, Yunfan Xiong, Yi Qian, Jiaqi Zhu, Shirong Ma, Xiaokang Zhang, Jiasheng Ye, et al. DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation. Technical report, DeepSeek-AI, 2026. Code and paper: https://github.com/deepseek-ai/DeepSpec.

[5] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2208.07339.

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.14314.

[7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1706.04599.

[8] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, 2023. arXiv:2211.17192.

[9] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.15077.

[10] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2406.16858.

[11] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.

[12] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*. MIT Press, 1999.

[13] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[14] Yudi Zhang, Weilin Zhao, Xu Han, Tiejun Zhao, Wang Xu, Hailong Cao, and Conghui Zhu. Speculative decoding meets quantization: Compatibility evaluation and hierarchical framework design. *arXiv preprint arXiv:2505.22179*, 2025.